ENTERPRISE SAAS LOYALTY MANAGEMENT PLATFORM

Business Requirements Document (BRD)
&
Software Requirements Specification (SRS)

Version	Date	Status	Classification
1.0.0	June 2025	DRAFT — FOR REVIEW	CONFIDENTIAL

Prepared by: Senior Enterprise Solutions Architect & Lead Product Manager
Intended Audience: Engineering, Product, Architecture, Executive Leadership

© 2025 LoyaltyOS Inc. All rights reserved.

Table of Contents

1 EXECUTIVE SUMMARY & PROJECT SCOPE 3
1.1 Vision & Objectives 3
1.2 Business Problem Statement 3
1.3 Scope Definition 3
2 ARCHITECTURAL AMBIGUITIES — RESOLVED 4
2.1 Dynamic Configuration & Multi-Tenancy 4
2.2 Data Ingestion Strategy 5
2.3 Rule Engine Design 5
2.4 Reward Engine Flow 6
2.5 AI/ML/LLM Integration Strategy 7
3 SYSTEM ARCHITECTURE (HIGH-LEVEL DESIGN) 8
3.1 Architecture Principles 8
3.2 Logical Architecture Overview 8
3.3 Component Breakdown 9
3.4 Technology Stack Justification 10
3.5 Database Strategy 11
3.6 Kafka & Event-Driven Architecture 12
4 CORE SYSTEM FLOWS 13
4.1 Data Ingestion Flow 13
4.2 Rule Evaluation Flow 13
4.3 Reward Issuance Flow 14
4.4 End-to-End Sequence Diagram (Descriptive) 14
5 FUNCTIONAL REQUIREMENTS 15
5.1 Business Portal & Tenant Onboarding 15
5.2 Rule Engine — Functional Requirements 17
5.3 Reward Engine — Functional Requirements 19
5.4 Customer-Facing APIs & Engagement 20
6 NON-FUNCTIONAL REQUIREMENTS 21
6.1 Scalability & Performance 21
6.2 Security & Compliance 22
6.3 Multi-Tenancy Constraints 23
6.4 Availability & Reliability 23
6.5 Observability & Operations 24

© 2025 LoyaltyOS Inc. All rights reserved.

7 MODERN TECH STACK ALIGNMENT (2024/2025) 25
8 RISK REGISTER 27
9 GLOSSARY 28

© 2025 LoyaltyOS Inc. All rights reserved.

§1 EXECUTIVE SUMMARY & PROJECT SCOPE

1.1 Vision & Objectives

This document defines the complete business and technical requirements for the LoyaltyOS Platform — a cloud-native, multi-tenant SaaS solution enabling any business, regardless of vertical (retail, gaming, e-commerce, fintech, hospitality), to design, deploy, and dynamically operate custom loyalty programs without requiring code changes.

Strategic Vision

LoyaltyOS removes the barrier between a business's loyalty idea and its live execution. Any tenant should be
able to onboard, configure their reward taxonomy, define behavioral rules, and start rewarding their customers
within hours — without writing a single line of code or engaging an engineering team.

Primary Objectives

- Deliver a zero-code, configuration-driven loyalty program builder accessible via a Business Portal.
- Process high-throughput event streams (customer transactions, behavioral signals) in near real-time.
- Execute a dynamic, tenant-configurable Rule Engine that evaluates events against complex business logic.
- Issue rewards (points, tiers, badges, vouchers, NFTs) reliably and idempotently via the Reward Engine.
- Support 10,000+ concurrent tenants and 100M+ end-user profiles with strict data isolation.
- Embed AI/ML capabilities for churn prediction, personalized reward recommendations, and LLM-assisted rule authoring.

1.2 Business Problem Statement

Existing loyalty solutions are either hyper-rigid (single-industry, non-configurable) or require significant professional services engagements to customize. SMBs and mid-market companies lack access to enterprise-grade loyalty infrastructure. Meanwhile, large enterprises waste engineering cycles rebuilding identical loyalty mechanics for every campaign.

Current Pain Points	LoyaltyOS Solution
12–18 week integration timelines for loyalty logic	Self-serve onboarding in < 4 hours via Business Portal
Rule changes require code deployments & QA cycles	No-code Rule Builder with hot-reload configuration

© 2025 LoyaltyOS Inc. All rights reserved.

Siloed per-tenant databases with no reusable infra	Shared infrastructure, strict tenant isolation via schemas/tags
No AI — static rules miss personalization opportunities	ML-driven churn prediction and personalized nudges

1.3 Scope Definition

In Scope — Release 1.0

- Multi-tenant onboarding and Business Portal (rule & reward configuration UI).
- Event Ingestion Layer: Webhook receivers, REST API push, Kafka consumer connectors.
- Rule Engine: SpEL-based expression evaluation with DB-persisted rule definitions.
- Reward Engine: Points ledger, tier management, badge issuance, basic voucher generation.
- Customer Profile Store and Loyalty Wallet APIs.
- Observability stack: distributed tracing, metrics, alerting.
- AI Module (Phase 1): Churn propensity scoring, LLM-assisted rule generation.

Out of Scope — Future Releases

- Native mobile SDKs (iOS / Android) — planned for R2.
- Blockchain / NFT-based reward tokens — planned for R3.
- Real-time fraud detection ML model — planned for R2.
- Direct POS hardware integrations — partner ecosystem roadmap.

© 2025 LoyaltyOS Inc. All rights reserved.

§2 ARCHITECTURAL AMBIGUITIES — RESOLVED

2.1 Dynamic Configuration & Multi-Tenancy

Problem Statement

How do we structure the system so that any business genre can dynamically configure loyalty rules without requiring code changes or redeployments?

Resolved Architecture: Configuration-as-Data with Hierarchical Tenant Model

The core principle is Configuration-as-Data: all tenant-specific business logic, reward schemas, event taxonomies, and rule definitions are stored as structured data in the platform's databases — never hardcoded in application logic. The application layer is a pure interpreter that reads and executes this configuration at runtime.

Tenant Isolation Model — Three-Layer Hierarchy

Layer 1 — Platform Level: Shared infrastructure (Kafka clusters, compute, networking).

Layer 2 — Tenant Level: Logical data isolation using PostgreSQL Row-Level Security (RLS) + tenant_id columns. Each tenant has its own Kafka consumer group and Redis namespace.

Layer 3 — Program Level: A single tenant may operate multiple loyalty programs (e.g., a retail chain running a VIP program AND a partner cashback program simultaneously).

Key Design Patterns

- Entity-Attribute-Value (EAV) + JSON Schema: Tenant-defined event schemas and reward attributes stored as JSONB in PostgreSQL, validated against a per-tenant JSON Schema at ingestion time.
- Feature Flags per Tenant: A TenantConfiguration document (MongoDB) controls which modules are active (e.g., tiers enabled, NFT rewards enabled, AI recommendations enabled).
- Dynamic Rule Schema: Rules are never compiled into JARs. They are persisted as structured rule definition objects (condition tree + action list) and interpreted at evaluation time by the Rule Engine.
- Hot-Reload Configuration Cache: Redis caches the tenant's active rule set with a configurable TTL. Rule updates propagate via a Kafka config-update topic, ensuring all Rule Engine pods reload within seconds without restart.

Multi-Tenancy Strategy

Concern	Approach	Justification
Database Isolation	Shared DB, separate schemas (PostgreSQL)	Balance of isolation vs. infrastructure cost; RLS enforces row-level security

© 2025 LoyaltyOS Inc. All rights reserved.

Kafka Isolation	Per-tenant consumer groups + topic naming convention: {tenantId}.events.*	Prevents cross-tenant message processing; enables per-tenant lag monitoring
Cache Isolation	Redis namespace prefix: tenant:{tenantId}:*	Simple, zero-overhead isolation in a shared Redis cluster
Object Storage	S3 bucket-level ACLs + tenant-prefixed key paths	Prevents data leakage for exported reports and uploaded assets

2.2 Data Ingestion Strategy

Decision: Push-Based Architecture with Webhook Ingestion as Primary Channel

After evaluating pull vs. push strategies, the platform adopts a push-first model. Requiring the platform to poll clients' APIs introduces coupling, credential management overhead, variable latency, and rate-limit dependencies. Instead, clients push events to us, enabling decoupled, real-time, scalable ingestion.

Ingestion Channels (Priority Order)

1. Webhook Push (Primary): Tenants configure a webhook endpoint in the Business Portal. Their backend systems POST structured event payloads (JSON) to https://ingest.loyaltyos.io/v1/{tenantId}/events. Payloads are validated against the tenant's JSON Schema, enriched with metadata, and published to Kafka.
2. REST API Push (Secondary): A synchronous REST endpoint for lower-volume, transactional event submission where the caller needs an immediate acknowledgement (e.g., point-of-sale systems requiring instant confirmation).
3. Kafka-Native Integration (Enterprise Tier): Large enterprise tenants with their own Kafka infrastructure can connect via Kafka MirrorMaker 2 for cross-cluster replication — eliminating HTTP overhead entirely.
4. SDK-Embedded Events (Future R2): Client-side JavaScript / mobile SDKs will instrument behavioral events (page views, app sessions) and batch-push to the ingestion layer.

Why NOT Pull-Based?

Pull requires storing OAuth tokens / API keys for hundreds of tenants — a significant secrets management and
security surface area. It introduces polling latency (typically 5–60 second intervals), requires the platform
to understand each client's API schema, and creates tight coupling. Push-based with a well-defined webhook
contract is the industry standard (Stripe, Shopify, Twilio all use this pattern).

2.3 Rule Engine Design

Decision: Hybrid SpEL + AST-Backed Custom Rule Engine (DB-Driven)

© 2025 LoyaltyOS Inc. All rights reserved.

Three options were evaluated: (1) Drools / Kogito, (2) a fully custom AST interpreter, (3) Spring Expression Language (SpEL) with a DB-persisted rule model. The recommendation is option 3 with a structured condition tree (AST-like) backing.

Why Not Drools?

- Drools is a full BRMS (Business Rules Management System) — significant operational overhead, Kie Server deployments, and DRL syntax complexity that non-technical tenant admins cannot use.
- Drools rules are files (DRL/XLS) — not first-class DB entities, making dynamic multi-tenant rule management complex.
- Boot time and memory footprint of the Drools engine per tenant context is prohibitive at scale.

Recommended Architecture: Structured Condition Tree + SpEL Evaluation

Rules are stored in PostgreSQL as a structured JSON document representing a condition tree (analogous to an AST). The Rule Engine deserializes this tree, constructs a SpEL expression, and evaluates it against the enriched event context.

Rule Data Model (Conceptual)

```
{
  ruleId: 'rule_001', tenantId: 'tenant_abc', name: 'Triple Points Weekend',
  priority: 10, status: 'ACTIVE',
  trigger: { eventType: 'PURCHASE' },
  conditions: {
    operator: 'AND',
    nodes: [
      { field: 'event.amount', operator: 'GTE', value: 500 },
      { field: 'event.dayOfWeek', operator: 'IN', value: ['SATURDAY','SUNDAY'] },
      { field: 'customer.tier', operator: 'EQ', value: 'GOLD' }
    ]
  },
  actions: [{ type: 'AWARD_POINTS', formula: 'event.amount * 3 * tenant.pointsRate' }]
}
```

Evaluation Process

5. Event arrives at Rule Engine service with enriched context object (event payload + customer profile + tenant config).
6. Rule Engine queries Redis cache for the tenant's active, prioritized rule set (TTL-based with Kafka invalidation).
7. For each eligible rule (matching eventType), the condition tree is traversed and a SpEL expression string is synthesized dynamically.
8. SpEL's StandardEvaluationContext is populated with the event context, and the expression is evaluated. SpEL's built-in functions handle arithmetic, date operations, and collection operations.

© 2025 LoyaltyOS Inc. All rights reserved.

9. All rules passing evaluation produce an ActionList. The Action Dispatcher resolves these to Reward Commands.

Rule Conflict Resolution

- Priority Field: Rules are ranked by an integer priority. In case of conflicting actions, higher-priority rules win.
- Execution Mode: Tenants configure FIRST_MATCH (stop after first passing rule) or ALL_MATCHING (accumulate all rewards) per rule group.
- Mutual Exclusion Tags: Rules can be tagged; only one rule per tag group executes per event evaluation.

Advanced Conditions Supported

Condition Type	Example	Implementation
Threshold	Purchase amount ≥ $500	SpEL: event.amount >= 500
Frequency	5th purchase this month	Redis counter: INCR + TTL
Temporal	Weekend or holiday	SpEL T(DayOfWeek) + calendar API
Compound	Gold tier AND first purchase	AND/OR operator tree evaluation
Streak	3 consecutive daily logins	Redis sorted set with expiry
Cumulative	Lifetime spend ≥ $10,000	PostgreSQL aggregate query

2.4 Reward Engine Flow

Conceptual Model: The Reward Command Pattern

The Rule Engine's output is not a direct database write — it is a Reward Command: a structured, idempotent instruction passed to the Reward Engine via Kafka. This decoupling is critical. It means the Rule Engine and Reward Engine scale independently, and reward issuance is resilient to downstream failures.

Reward Types Supported

Reward Type	Storage	Consistency Model	Expiry Support
Points	PostgreSQL Ledger (append-only)	Serializable transactions	Yes — TTL on ledger entries
Tier Status	Customer Profile (MongoDB)	Optimistic locking	Yes — annual reset
Badges / Achievements	Customer Profile (MongoDB)	Idempotent upsert	No (permanent)

© 2025 LoyaltyOS Inc. All rights reserved.

Vouchers / Coupons	Voucher Service (PostgreSQL)	Unique code generation	Yes — configurable
Cashback	Ledger + external payout integration	Two-phase commit (saga)	No

Points Ledger Design — Immutable Append-Only Pattern

The points balance is NEVER stored as a single mutable counter. Instead, every point transaction (credit or debit) is recorded as an immutable ledger entry. The current balance is the SUM of all non-expired ledger entries. This provides a complete audit trail, supports point expiry by TTL per entry, and eliminates concurrent update race conditions.

Ledger Entry Schema

```
ledger_entry: { id, tenant_id, customer_id, transaction_id (idempotency key),
  type: CREDIT | DEBIT | EXPIRE | ADJUST,
  points: decimal, source_rule_id, source_event_id,
  expires_at: timestamp | null, created_at, metadata: jsonb }
```

Idempotency & Security

- Every Reward Command carries a unique idempotency_key (SHA-256 hash of tenantId + eventId + ruleId). Duplicate commands are detected via a Redis idempotency store (TTL 24h) before processing.
- Tier upgrades trigger a Tier Evaluation Job that re-evaluates the customer's lifetime metrics against the tenant's tier thresholds — preventing both over- and under-promotion.
- All reward mutations are published as RewardIssued events to Kafka for downstream consumption (notifications, analytics, audit log).

2.5 AI/ML/LLM Integration Strategy

Practical Use Cases — Prioritized by Value & Implementation Complexity

Use Case	Description	Model Type	Release
LLM Rule Generation	Business admin describes rule in plain English; LLM converts to structured rule JSON	GPT-4o / Claude 3.5	R1
Churn Propensity	XGBoost model predicts customers at risk of disengagement; triggers proactive reward nudges	XGBoost / LightGBM	R1
Personalized Rewards	Collaborative filtering recommends the reward type most likely to drive re-engagement per customer	Matrix Factorization	R2

© 2025 LoyaltyOS Inc. All rights reserved.

Tier Trajectory Forecast	Time-series model predicts whether a customer will reach the next tier — enables targeted communications	Prophet / LSTM	R2
Anomaly Detection	Isolation Forest detects unusual reward accumulation patterns (fraud / gaming the system)	Isolation Forest	R2
LLM Analytics Assistant	NL-to-SQL interface for business admins to query loyalty metrics without knowing SQL	GPT-4o + LangChain	R3

LLM Rule Generation — Technical Detail

This is the highest-ROI AI feature for R1. The Business Portal includes a natural language input field. When a tenant admin types 'Award double points on Saturdays for customers who have made at least 3 purchases this month,' the system:

10. Sends the text to an LLM (GPT-4o / Claude) with a system prompt containing the tenant's event schema and rule JSON format specification.
11. The LLM returns a structured rule JSON object.
12. The portal renders this as a visual rule card for admin review and confirmation.
13. On confirmation, the rule is saved to the database — no code change, no engineering involvement.

© 2025 LoyaltyOS Inc. All rights reserved.

§3 SYSTEM ARCHITECTURE (HIGH-LEVEL DESIGN)

3.1 Architecture Principles

- Event-Driven First: All inter-service communication for non-synchronous workflows uses Kafka. HTTP/gRPC is reserved for synchronous, low-latency queries.
- Immutability of State Mutations: Ledger entries, event logs, and audit records are append-only. No hard deletes.
- Zero Trust Security: Every service-to-service call is authenticated via mTLS + JWT. No implicit trust within the cluster.
- Configuration Over Code: All tenant behavior is expressed as data, not code. The platform is an interpreter, not a compiler.
- Idempotency Everywhere: All Kafka consumers and API endpoints are designed to handle duplicate messages safely.
- Polyglot Persistence: The right database for the right job — no single DB for all concerns.

3.2 Logical Architecture Overview

Five Logical Layers

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ LAYER 5: INTELLIGENCE PLANE                                                  │
│ ML Feature Store │ Churn Scorer │ Recommendation Engine │ LLM API           │
├──────────────────────────────────────────────────────────────────────────────┤
│ LAYER 4: PROCESSING PLANE                                                    │
│ Rule Engine Service │ Reward Engine Service │ Tier Evaluation Job            │
├──────────────────────────────────────────────────────────────────────────────┤
│ LAYER 3: STREAMING PLANE (Apache Kafka)                                      │
│ Ingestion Topics │ Rule-Eval Topics │ Reward-Command Topics                 │
│ Config-Update Topics │ Audit Topics │ Notification Topics                   │
├──────────────────────────────────────────────────────────────────────────────┤
│ LAYER 2: DATA PLANE                                                          │
│ PostgreSQL (ledger, rules) │ MongoDB (profiles) │ Redis (cache)             │
│ Neo4j (social graph) │ ClickHouse (analytics) │ S3 (objects)               │
├──────────────────────────────────────────────────────────────────────────────┤
│ LAYER 1: INGESTION / API PLANE                                               │
│ Webhook Receiver │ REST Ingestion API │ Business Portal API                 │
│ Customer API │ API Gateway (Kong) │ CDN (Cloudflare)                        │
└──────────────────────────────────────────────────────────────────────────────┘
```

© 2025 LoyaltyOS Inc. All rights reserved.

3.3 Component Breakdown

3.3.1 Ingestion Plane

- Webhook Receiver (Spring Boot): Validates HMAC signature, authenticates tenant, deserializes payload, validates against JSON Schema, enriches with metadata (tenantId, timestamp, ingestionId), publishes to Kafka topic {tenantId}.events.raw.
- REST Ingestion API: Synchronous endpoint for transactional events. Returns HTTP 202 Accepted after Kafka publish. No synchronous rule evaluation.
- API Gateway (Kong): Rate limiting, JWT authentication, request routing, tenant identification from API key headers, SSL termination.

3.3.2 Streaming Plane (Kafka Topics)

Topic Pattern	Partitions	Purpose
{tenantId}.events.raw	32 (auto-scaled)	Raw inbound events from all ingestion channels
{tenantId}.events.enriched	32	Events enriched with customer profile data
platform.rule.eval	64	Events queued for rule evaluation (all tenants)
platform.reward.commands	64	Reward commands from Rule Engine to Reward Engine
platform.rewards.issued	32	Confirmation events for issued rewards (fanout)
platform.config.updates	8	Rule / config change notifications for cache invalidation
platform.audit.log	16	Immutable audit trail — all mutations
platform.notifications	16	Outbound notification triggers (email, push, webhook)

3.3.3 Rule Engine Service (Spring Boot)

- Consumes from platform.rule.eval topic (Kafka consumer group: rule-engine-cg).
- Loads tenant rule set from Redis cache (L1) or PostgreSQL (L2 on cache miss).
- Executes SpEL-based condition evaluation against enriched event context.
- Publishes zero-to-many RewardCommand messages to platform.reward.commands.
- Horizontally scalable — stateless pods; all state in Kafka + Redis.

3.3.4 Reward Engine Service (Spring Boot)

- Consumes from platform.reward.commands with exactly-once semantics (Kafka transactions).
- Enforces idempotency via Redis-based deduplication before any DB write.

© 2025 LoyaltyOS Inc. All rights reserved.

- Executes reward issuance: ledger writes (PostgreSQL), profile updates (MongoDB), badge grants.
- Publishes RewardIssued confirmation events for downstream consumers (notifications, analytics).

3.3.5 Business Portal (React + Next.js 14)

- Tenant onboarding wizard (5-step guided setup).
- Visual Rule Builder: drag-and-drop condition tree with LLM assist input.
- Reward Catalog manager: define points schemas, tier thresholds, badge criteria.
- Analytics dashboard: powered by ClickHouse queries via a dedicated Analytics API.
- Customer lookup and loyalty wallet viewer for tenant support teams.

3.4 Technology Stack Justification

Component	Technology	Justification
Core Services	Spring Boot 3.3 (Java 21)	Virtual threads (Project Loom) for high-concurrency I/O. Battle-tested enterprise ecosystem. Native compilation via GraalVM for cold-start optimization.
API Gateway	Kong Gateway 3.x	Lua plugin extensibility, rate limiting, JWT validation, tenant routing, OpenTelemetry native support.
Event Streaming	Apache Kafka (Confluent Platform / MSK)	Industry-standard event streaming. Supports exactly-once semantics, 100K+ TPS per partition, Kafka Streams for in-flight aggregations.
Transactional DB	PostgreSQL 16 (CockroachDB for geo-dist.)	ACID compliance for ledger and rule data. JSONB support for schema-flexible EAV attributes. Row-Level Security for tenant isolation.
Document Store	MongoDB 7 Atlas	Flexible schema for heterogeneous customer profiles and tenant configurations. Atlas Search for full-text lookup.
Cache / Session	Redis 7.2 (Redis Cluster)	Sub-millisecond rule cache lookups. Sorted sets for frequency counters and streak tracking. Pub/Sub for config invalidation.
Analytics OLAP	ClickHouse (self-hosted or Cloud)	Columnar storage purpose-built for analytical queries. Handles billions of event rows with sub-second aggregation. Kafka connector for real-time ingestion.
Graph DB	Neo4j (optional — R2)	Customer social graph for referral tracking and viral reward programs. Cypher queries for multi-hop relationship traversal.
Search	Elasticsearch 8 / OpenSearch	Customer profile search, reward catalog full-text search, and audit log exploration.

© 2025 LoyaltyOS Inc. All rights reserved.

Object Storage	AWS S3 / GCS	Report exports, batch import files, tenant asset uploads. Lifecycle policies for cost management.
Frontend	Next.js 14 + React 18 + Tailwind	App Router for server components, RSC for initial render performance, Tailwind for design system consistency.
ML Platform	MLflow + Feast + SageMaker	MLflow for experiment tracking and model registry. Feast for feature store (online + offline). SageMaker for scalable inference endpoints.
Container Orch.	Kubernetes (EKS / GKE) + Helm	KEDA for Kafka-driven autoscaling of Rule and Reward Engine pods. HPA for API services.
Service Mesh	Istio + Envoy	mTLS between all services, traffic shaping, circuit breaking, distributed tracing integration.
Observability	Grafana + Prometheus + Tempo + Loki	Full LGTM stack: Loki (logs), Grafana (dashboards), Tempo (traces), Mimir (metrics at scale). OpenTelemetry for instrumentation.
CI/CD	GitHub Actions + ArgoCD	GitOps workflow with ArgoCD for Kubernetes deployments. Automated vulnerability scanning (Trivy) in pipeline.

3.5 Database Strategy — Polyglot Persistence Rationale

Why Multiple Databases?

Each database is chosen for a specific access pattern. A single 'universal' database inevitably compromises on at least one critical requirement — typically either transactional integrity or query flexibility.

Database	Stores	Access Pattern	Key Requirement Met
PostgreSQL 16	Points ledger, rule definitions, tenant config, vouchers	ACID txn, point-in-time queries, audit	Financial-grade consistency for ledger; RLS for tenant isolation
MongoDB Atlas	Customer profiles, loyalty wallets, tenant feature flags	Document read/write, flexible schema evolution	Heterogeneous profile attributes across industry verticals
Redis 7.2	Rule cache, idempotency keys, frequency counters, sessions, streak data	Sub-ms reads, atomic increments, TTL management	Rule Engine performance (<5ms rule load), idempotency at scale
ClickHouse	All platform events, reward issuance history, analytics aggregates	Bulk insert, columnar analytical queries	Real-time analytics dashboard queries across billions of events
Neo4j	Referral trees, social connections, program partnerships	Graph traversal, shortest-path, community detection	Referral bonus attribution across multi-hop chains

© 2025 LoyaltyOS Inc. All rights reserved.

3.6 Kafka & Event-Driven Architecture — Deep Dive

Why Kafka Over Alternatives (RabbitMQ, SQS, Pub/Sub)?

- Kafka provides durable, replayable, ordered event logs — critical for the audit trail and for reprocessing events after rule configuration changes.
- Kafka's consumer group model allows horizontal scaling of Rule Engine pods without message duplication.
- Kafka's exactly-once semantics (EOS) via transactional producers and consumers is essential for the Reward Engine — preventing double reward issuance.
- Kafka Streams enables stateful in-stream aggregations (e.g., rolling purchase totals for frequency-based rules) without an external database round-trip.
- Kafka Connect provides out-of-the-box connectors to PostgreSQL (Debezium CDC), ClickHouse, MongoDB, and S3 — eliminating custom ETL code.

Consumer Lag as Autoscaling Signal

KEDA (Kubernetes Event-Driven Autoscaler) monitors Kafka consumer group lag on the platform.rule.eval and platform.reward.commands topics. When lag exceeds configurable thresholds (e.g., 10,000 uncommitted messages), KEDA automatically scales the Rule Engine and Reward Engine pod count. During off-peak hours, pods scale down to zero — eliminating idle compute costs.

© 2025 LoyaltyOS Inc. All rights reserved.

§4 CORE SYSTEM FLOWS

4.1 Data Ingestion Flow

Step-by-Step: Webhook Event Ingestion

14. Tenant backend POSTs an event to https://ingest.loyaltyos.io/v1/{tenantId}/events with an HMAC-SHA256 signature in the X-LoyaltyOS-Signature header.
15. Kong API Gateway validates the API key, identifies the tenantId, enforces rate limits (configurable per tenant tier), and routes to the Webhook Receiver service.
16. Webhook Receiver validates the HMAC signature using the tenant's stored webhook secret (retrieved from HashiCorp Vault). Invalid signatures return HTTP 401.
17. Payload is validated against the tenant's registered JSON Schema (cached in Redis). Schema violations return HTTP 422 with structured error details.
18. Valid payload is enriched with metadata: ingestion_id (UUID v7), tenant_id, received_at (UTC), source_channel: WEBHOOK.
19. Enriched event is published to the Kafka topic {tenantId}.events.raw. HTTP 202 Accepted returned to the caller immediately — no synchronous processing.
20. A Kafka Streams enrichment topology consumes from {tenantId}.events.raw, fetches the customer profile from MongoDB (by customer_id in the payload), merges profile data into the event, and publishes to platform.rule.eval.

4.2 Rule Evaluation Flow

Step-by-Step: Rule Engine Processing

21. Rule Engine pod consumes an enriched event from platform.rule.eval (assigned partition by consumer group).
22. Extract tenantId from event. Check Redis for tenant's active rule set (key: tenant:{tenantId}:rules:active). On cache hit, deserialize. On cache miss, query PostgreSQL for active rules ordered by priority, populate Redis cache (TTL: 300s).
23. Filter rules by trigger.eventType matching event.eventType. Skip rules with non-matching triggers.
24. For each candidate rule, traverse the condition tree node by node. For each leaf node, construct a SpEL expression fragment. Combine with AND/OR operators per the tree structure. Examples: event.amount >= 500 AND customer.tier == 'GOLD'.
25. Evaluate the composite SpEL expression using StandardEvaluationContext populated with: {event: eventPayload, customer: profileData, tenant: tenantConfig, now: Instant.now()}.
26. For rules requiring frequency checks (e.g., '5th purchase this month'), execute a Redis INCR on key tenant:{tenantId}:freq:{customerId}:{ruleId}:{yearMonth} with TTL set to end-of-month. Compare result against the rule's threshold.
27. Collect all passing rules. Apply execution mode logic: FIRST_MATCH stops after the highest-priority match; ALL_MATCHING collects all passing actions.
28. For each passing rule's action list, construct a RewardCommand message: {commandId, idempotencyKey, tenantId, customerId, ruleId, eventId, actionType, parameters, issuedAt}.

© 2025 LoyaltyOS Inc. All rights reserved.

Publish all RewardCommands to platform.reward.commands via a Kafka transactional producer.

29. Mark the event as PROCESSED in the audit log topic platform.audit.log.

4.3 Reward Issuance Flow

Step-by-Step: Reward Engine Processing

30. Reward Engine pod consumes a RewardCommand from platform.reward.commands.
31. Check idempotency: SETNX tenant:{tenantId}:idem:{idempotencyKey} in Redis with TTL 86400 (24h). If key already exists, discard the command (duplicate) and commit Kafka offset. Log as DUPLICATE_SKIPPED.
32. Begin a PostgreSQL serializable transaction.
33. Route by actionType: AWARD_POINTS → compute points amount using the rule's formula (SpEL evaluated with event context) → insert a CREDIT ledger entry into the points_ledger table. UPGRADE_TIER → call Tier Evaluation Service to validate tier eligibility → update customer profile in MongoDB. GRANT_BADGE → upsert badge into customer's badge collection in MongoDB with idempotent check. ISSUE_VOUCHER → generate a unique voucher code → insert into vouchers table with status ISSUED.
34. Commit the PostgreSQL transaction. On commit success, commit the Kafka consumer offset (exactly-once via Kafka transactions).
35. Publish a RewardIssued event to platform.rewards.issued for downstream consumers: Notification Service (sends email/push), Analytics Sink (ClickHouse ingestion), Customer API cache invalidation.
36. On PostgreSQL transaction failure (deadlock, constraint violation), apply exponential backoff retry (3 attempts). After max retries, publish to platform.reward.commands.DLQ (Dead Letter Queue) and alert the on-call SRE.

4.4 End-to-End Sequence Summary

Complete Flow: PURCHASE Event → Triple Points Reward

```
[1] Client Backend  ──POST /events──>  [2] Kong Gateway  ──route──>  [3] Webhook Receiver
                                                  │
                                            validate + publish
                                                  │
[4] Kafka: {tenantId}.events.raw  ──stream──>  [5] Enrichment Topology (Kafka Streams)
                                    │  (merge customer profile)
                                    │
[6] Kafka: platform.rule.eval  ──consume──>  [7] Rule Engine Pod
                                  │  load rules from Redis
                                  │  evaluate SpEL conditions
                                  │  rule passes → build RewardCommand
                                  │
[8] Kafka: platform.reward.commands  ──consume──>  [9] Reward Engine Pod
                                    │  idempotency check (Redis)
                                    │  write ledger (PostgreSQL ACID)
                                    │
```

© 2025 LoyaltyOS Inc. All rights reserved.

```
[10] Kafka: platform.rewards.issued  ──fanout──>  [11a] Notification Service
                                                  [11b] ClickHouse Analytics Sink
                                                  [11c] Customer API Cache Invalidation

Total P99 latency target: < 2 seconds (event receipt → reward issued)
```

© 2025 LoyaltyOS Inc. All rights reserved.

§5 FUNCTIONAL REQUIREMENTS

Requirement Priority Legend
P0 = Must Have (MVP blocker) P1 = Should Have (R1) P2 = Nice to Have (R2+)

5.1 Business Portal & Tenant Onboarding

FR-BP-001 Tenant Self-Serve Onboarding

ID	Requirement
BP-001	P0 — The system shall provide a guided 5-step onboarding wizard: (1) Account Setup, (2) Industry/Vertical Selection, (3) Event Schema Definition, (4) Reward Catalog Setup, (5) Webhook Configuration. Tenant should be able to go live within 4 hours.
BP-002	P0 — The system shall generate a unique tenantId, API keys (rotatable), and a webhook endpoint URL upon tenant registration.
BP-003	P0 — The portal shall support role-based access control (RBAC) within a tenant: ADMIN (full access), PROGRAM_MANAGER (rules + rewards), ANALYST (read-only analytics), SUPPORT (customer lookup).
BP-004	P1 — The system shall support multi-program management — a single tenant can create and manage multiple loyalty programs, each with independent rule sets and reward catalogs.
BP-005	P1 — The portal shall support white-label branding: custom logo, primary/secondary color scheme, and email template customization.

FR-BP-002 Event Schema Management

ID	Requirement
BP-010	P0 — The portal shall allow tenant admins to define custom event types (e.g., PURCHASE, CHECKIN, GAME_LEVEL_COMPLETED) with associated field schemas (field name, type, required/optional, validation constraints).
BP-011	P0 — The system shall provide a library of pre-built event templates for common industry verticals (Retail, Gaming, E-commerce, Hospitality) that tenants can adopt and customize.
BP-012	P1 — Tenant admins shall be able to test their event schema by submitting a sample payload through the portal and seeing real-time validation results.
BP-013	P1 — Schema versioning: the system shall maintain a version history of event schemas. Changing a schema shall not break ingestion of events conforming to older schema versions (backward compatibility window: 30 days).

© 2025 LoyaltyOS Inc. All rights reserved.

FR-BP-003 Analytics Dashboard

ID	Requirement
BP-020	P0 — The portal shall display real-time (< 30s latency) metrics: total active members, points issued today, top rules triggered, tier distribution breakdown.
BP-021	P1 — The portal shall provide cohort analysis: new vs. returning members, engagement by tier, and reward redemption rates over configurable time windows.
BP-022	P1 — All dashboard data shall be exportable to CSV/XLSX with selectable date ranges.
BP-023	P2 — LLM-powered Analytics Assistant: tenant admins can ask questions in plain English (e.g., 'Which customers are at risk of disengaging in the next 30 days?') and receive natural-language answers with supporting data.

5.2 Rule Engine — Functional Requirements

FR-RE-001 Rule Lifecycle Management

ID	Requirement
RE-001	P0 — The system shall allow tenant admins to CREATE, READ, UPDATE, DELETE, and CLONE loyalty rules through the Business Portal with no code changes required.
RE-002	P0 — Rules shall support status lifecycle: DRAFT → ACTIVE → PAUSED → ARCHIVED. Only ACTIVE rules are evaluated. Status transitions require explicit confirmation.
RE-003	P0 — Rule activation/deactivation shall propagate to all Rule Engine pods within ≤ 10 seconds via Kafka config-update topic and Redis cache invalidation.
RE-004	P0 — All rule changes (create, update, status change, delete) shall be recorded in an immutable audit log with the admin's identity, timestamp, and before/after state.
RE-005	P1 — Rules shall support scheduled activation windows: start_date/end_date for time-limited campaigns (e.g., holiday double-points promotion).
RE-006	P1 — The system shall enforce a per-tenant rule limit based on subscription tier (Standard: 50 active rules, Professional: 500, Enterprise: unlimited).

FR-RE-002 Condition & Action Types

ID	Requirement
RE-010	P0 — The Rule Engine shall support the following condition operators: EQ, NEQ, GT, GTE, LT, LTE, IN, NOT_IN, CONTAINS, STARTS_WITH, BETWEEN, IS_NULL, IS_NOT_NULL.
RE-011	P0 — Conditions shall be composable with AND/OR/NOT logical operators, supporting unlimited nesting depth.

© 2025 LoyaltyOS Inc. All rights reserved.

RE-012	P0 — The following action types shall be supported: AWARD_POINTS (with formula), AWARD_BONUS_POINTS (flat amount), GRANT_BADGE, ISSUE_VOUCHER, TRIGGER_NOTIFICATION, WEBHOOK_CALLBACK (to tenant endpoint).
RE-013	P1 — Point award formulas shall support arithmetic expressions referencing event fields, customer profile attributes, and tenant constants. Examples: event.amount * 0.02, customer.lifetimeValue * tenantConfig.pointsMultiplier.
RE-014	P1 — The Rule Engine shall support frequency-based conditions: 'event is the Nth occurrence of type X for this customer within time window W'. Counters shall be maintained in Redis with automatic TTL expiry.
RE-015	P1 — The Rule Engine shall support streak conditions: 'customer has performed action X for N consecutive days/weeks'. Streak state shall be maintained in Redis sorted sets.

FR-RE-003 LLM-Assisted Rule Generation

ID	Requirement
RE-020	P1 — The Rule Builder UI shall include a plain-English input: 'Describe your rule.' The system shall invoke the LLM API with the tenant's event schema context and return a pre-filled rule form for admin review.
RE-021	P1 — LLM-generated rules shall NEVER be auto-activated. They shall always enter DRAFT status and require explicit admin review and activation.
RE-022	P1 — The system shall maintain a prompt template library per industry vertical to improve LLM rule generation accuracy. Templates shall be versioned and A/B tested.
RE-023	P2 — LLM rule generation shall support multi-turn conversation: the admin can ask follow-up questions ('What if the customer is in the Silver tier instead?') and the system shall update the rule draft accordingly.

5.3 Reward Engine — Functional Requirements

FR-RW-001 Points Management

ID	Requirement
RW-001	P0 — The system shall maintain an immutable, append-only points ledger per customer per tenant. Balance is always derived from SUM of non-expired CREDIT entries minus SUM of DEBIT entries.
RW-002	P0 — Points issuance shall be idempotent. Reprocessing the same event shall not result in duplicate points. Idempotency enforced via Redis SETNX before any ledger write.
RW-003	P0 — The system shall support configurable points expiry: global tenant-level TTL (e.g., points expire 12 months after issuance) or per-campaign TTL.

© 2025 LoyaltyOS Inc. All rights reserved.

RW-004	P0 — Points redemption shall atomically debit the ledger and decrement the customer's available balance. Insufficient balance shall return a structured error; no negative balance permitted by default.
RW-005	P1 — Tenant admins shall be able to issue manual point adjustments (CREDIT or DEBIT) for customer service purposes. Manual adjustments require a reason code and are separately identifiable in the audit log.

FR-RW-002 Tier Management

ID	Requirement
RW-010	P0 — Tenants shall be able to define up to 10 custom tiers with configurable names, qualification criteria (e.g., lifetime spend, points in rolling 12 months, transaction count), and associated benefits.
RW-011	P0 — Tier qualification shall support multiple criteria types: lifetime cumulative (never decreases), rolling period (recalculated on a schedule), and transaction count.
RW-012	P0 — Tier evaluation shall run asynchronously after each qualifying event. Near-real-time tier upgrades (target: < 5 seconds from event ingestion to tier status update).
RW-013	P1 — Grace period on tier downgrades: the system shall support a configurable 'tier protection window' (e.g., 3 months) during which a customer retains their tier even if qualification criteria are no longer met.

5.4 Customer-Facing APIs & Engagement

ID	Requirement
CA-001	P0 — Public REST API (v1): GET /customers/{customerId}/wallet — returns current points balance, tier status, badge collection, active vouchers. Response time target: < 100ms P99.
CA-002	P0 — GET /customers/{customerId}/transactions — paginated points history with filterable date range, transaction type, and source rule.
CA-003	P0 — POST /customers/{customerId}/redeem — submit a redemption request for points or voucher. Idempotent via redemption_reference field.
CA-004	P1 — GET /customers/{customerId}/progress — returns tier progress (current vs. required), next tier name, and projected attainment date (AI-driven forecast).
CA-005	P1 — GET /customers/{customerId}/recommendations — returns personalized reward recommendations ranked by ML model propensity score.
CA-006	P1 — Outbound Webhooks: the system shall call back the tenant's registered webhook URL when key events occur: RewardIssued, TierUpgraded, BadgeGranted, PointsExpiringSoon (configurable notice period).

© 2025 LoyaltyOS Inc. All rights reserved.

§6 NON-FUNCTIONAL REQUIREMENTS

6.1 Scalability & Performance

Metric	Target	Mechanism
Event Ingestion Throughput	500,000 events/second (platform-wide)	Kafka horizontal partitioning + stateless Webhook Receiver pods (KEDA)
Rule Evaluation Latency	< 20ms P99 per event	Redis rule cache, SpEL pre-compiled expressions, JVM JIT warm-up
Reward Issuance Latency	< 500ms P99 end-to-end	Kafka exactly-once, PostgreSQL connection pooling (PgBouncer), optimistic locking
Customer API Response Time	< 100ms P99	Redis wallet cache, MongoDB indexed reads, CDN for cacheable responses
Total End-to-End Latency	< 2 seconds P99	Full pipeline: event receipt → reward issued confirmation
Concurrent Tenants	10,000+	Shared infrastructure with logical isolation; per-tenant Kafka consumer groups
Customer Profiles	100M+ per tenant (large enterprise)	MongoDB sharding by customer_id; ClickHouse partitioning by tenant + date
Peak Traffic Handling	10x normal load (Black Friday, etc.)	KEDA autoscaling; Kafka consumer group rebalance; pre-warming strategy

Horizontal Scaling Strategy

- All processing services (Webhook Receiver, Rule Engine, Reward Engine) are stateless Spring Boot pods running in Kubernetes. All state is externalized to Kafka, Redis, PostgreSQL, and MongoDB.
- KEDA monitors Kafka consumer lag and scales Rule Engine pods between 2 (minimum) and 100 (maximum) based on configured lag thresholds.
- Database read scaling: MongoDB Atlas read replicas + PostgreSQL streaming replicas behind PgPool-II for read queries. Write path always goes to primary.
- ClickHouse scales horizontally for analytical query load, independent of transactional databases.

6.2 Security & Compliance

© 2025 LoyaltyOS Inc. All rights reserved.

Authentication & Authorization

- API Key Authentication for tenant webhook integrations. Keys are rotatable, scoped to specific permissions, and stored hashed in PostgreSQL (bcrypt).
- OAuth 2.0 + OIDC for Business Portal human users. MFA enforcement for ADMIN and PROGRAM_MANAGER roles. Integration with enterprise IdPs (Okta, Auth0, Azure AD) via SAML 2.0.
- Customer API tokens are short-lived JWTs (15-minute TTL) issued via the tenant's own authentication flow. The platform validates signature against the tenant's registered public key.
- Service-to-service authentication: Kubernetes service accounts + Istio mTLS. SPIFFE/SPIRE for workload identity.

Data Security

- Encryption at rest: AES-256 for all database volumes (EBS/GCS encryption). Application-level encryption for PII fields (name, email, phone) using envelope encryption with tenant-specific DEKs stored in HashiCorp Vault.
- Encryption in transit: TLS 1.3 for all external communications. mTLS between internal services via Istio.
- Data masking: Support staff and analytics queries apply automatic PII masking based on data classification tags. Raw PII only accessible by authorized roles with full audit logging.
- Secrets management: HashiCorp Vault for API keys, database credentials, webhook secrets, and LLM API keys. Dynamic secret rotation for database credentials.

Compliance

Standard	Requirement	Implementation
GDPR	Right to erasure; data portability; explicit consent for AI processing	Soft-delete with PII anonymization; data export API; consent flags in customer profile
CCPA	Right to opt-out of sale; data deletion	Tenant-level configuration for data residency and opt-out handling
SOC 2 Type II	Security, availability, and confidentiality controls	Annual audit; automated compliance evidence collection via Drata/Vanta
PCI DSS	If payment data in scope (cashback integrations)	Network segmentation; tokenization of payment references; no raw card data stored
ISO 27001	Information security management	Target certification within 18 months of platform launch

6.3 Multi-Tenancy Constraints

NFR-MT-001 Tenant Isolation Guarantees

- Hard guarantee: No tenant's data, rules, or customer profiles shall ever be accessible to another tenant's API calls or processing jobs. Enforced at: application layer (tenantId scoping on all

© 2025 LoyaltyOS Inc. All rights reserved.

queries), database layer (PostgreSQL RLS policies), and network layer (Kubernetes NetworkPolicies).

- Noisy Neighbor Prevention: Per-tenant rate limits enforced at Kong (requests per second per tenant). Kafka consumer groups are isolated per tenant, preventing one tenant's event burst from delaying another's processing.
- Resource quotas: Kubernetes ResourceQuotas and LimitRanges are applied per tenant namespace for enterprise deployments. Standard-tier tenants share namespace pools with resource limits.

NFR-MT-002 Data Residency

- Enterprise tenants may specify a data residency region (US, EU, APAC). The platform deploys regional PoPs for each region and ensures customer PII does not leave the specified region boundary.
- Metadata (aggregate metrics, non-PII analytics) may be processed globally for operational purposes.

6.4 Availability & Reliability

Service	SLA Target	Strategy
Ingestion API (Webhook Receiver)	99.99% (< 52 min/year)	Multi-AZ deployment; Circuit breaker (Resilience4J); health checks
Rule Engine	99.95% (< 4.4 hr/year)	KEDA autoscaling; Kafka-backed retry; DLQ for unprocessable messages
Reward Engine	99.99%	Exactly-once Kafka semantics; idempotent writes; multi-AZ PostgreSQL
Customer API	99.99%	Redis cache fallback; read replicas; CDN for cacheable responses
Business Portal	99.9% (< 8.7 hr/year)	CDN-hosted static assets; serverless API fallback for read-only views
Kafka (MSK/Confluent)	99.99% (provider SLA)	3-replica brokers; replication factor 3; min.insync.replicas = 2

Disaster Recovery

- RTO (Recovery Time Objective): 15 minutes for Reward Engine and Customer API. 4 hours for full platform recovery.
- RPO (Recovery Point Objective): < 30 seconds. Achieved through Kafka replication, PostgreSQL WAL streaming to standby, and MongoDB Atlas continuous backup.
- Chaos Engineering: Monthly GameDay exercises using Chaos Mesh to simulate pod failures, network partitions, and database failovers.

© 2025 LoyaltyOS Inc. All rights reserved.

6.5 Observability & Operations

Three Pillars of Observability

- Metrics (Prometheus + Grafana Mimir): Key metrics include Kafka consumer lag by topic/group, rule evaluation throughput/error rate, reward issuance success/failure rate, API latency percentiles (P50, P95, P99), database connection pool utilization. Tenant-scoped dashboards for business admins (events processed, rewards issued today).
- Logs (Loki + Grafana): Structured JSON logging from all services. Correlation IDs (eventId, tenantId, traceId) included in every log line. Retention: 30 days hot, 1 year cold (S3 archive). PII fields automatically redacted before log emission.
- Traces (Tempo + OpenTelemetry): Distributed traces across the full pipeline: Webhook Receiver → Kafka → Rule Engine → Kafka → Reward Engine → Database. P99 trace latency alerting. Automatic anomaly detection on trace duration spikes.

Alerting & On-Call

- PagerDuty integration for P0/P1 alerts: Reward Engine DLQ message count > 0, Kafka consumer lag > 10,000 for > 5 minutes, API error rate > 1%, Database replication lag > 30 seconds.
- Automated runbooks in Confluence triggered from Grafana alert annotations.

© 2025 LoyaltyOS Inc. All rights reserved.

§7 MODERN TECH STACK ALIGNMENT (2024/2025)

All technology choices reflect current enterprise adoption trends as of 2024–2025. The following table documents the industry alignment and maturity level for each component.

Technology	2024/25 Trend Alignment	Industry Adoption	Maturity
Java 21 + Virtual Threads	Project Loom GA — eliminates reactive complexity for high-concurrency	Netflix, Uber, LinkedIn migrating to Java 21 for throughput gains	GA — Production Ready
Spring Boot 3.3	Native GraalVM compilation, Micrometer Tracing, Spring AI integration	Dominant enterprise Java framework; 70%+ market share	GA — Production Ready
Apache Kafka (Confluent)	Kafka 3.x with KRaft (ZooKeeper eliminated); Confluent Cloud BYOC model	Standard for enterprise event streaming; LinkedIn, Airbnb, Uber	GA — Dominant Standard
KEDA (Kafka Autoscaling)	CNCF graduated project; de-facto standard for event-driven K8s scaling	Rapid adoption in 2023–2025; Azure Functions uses KEDA internally	CNCF Graduated
PostgreSQL 16 (CockroachDB)	PG 16 logical replication improvements; CockroachDB for global ACID	Most popular open-source RDBMS; CockroachDB for geo-distributed ACID	GA — Enterprise Standard
MongoDB Atlas	Atlas Vector Search (2024); Atlas Stream Processing for Kafka integration	60%+ of document DB market; Goldman Sachs, eBay, Adobe	GA — Production Ready
Redis 7.2 Cluster	Redis 7.2 multi-part AOF; Redis Cluster auto-scaling; Redis Stack (vector, search)	Industry standard for caching and session; Twitter, GitHub, Pinterest	GA — Dominant Standard
ClickHouse	ClickHouse Cloud GA 2023; 5–100x faster than Redshift for event analytics	Cloudflare, ByteDance, Uber for real-time analytics; fastest-growing OLAP	GA — High Growth
Next.js 14 App Router	Server Components GA; streaming SSR; Partial Pre-rendering (experimental)	Vercel, OpenAI, Twitch, TikTok; dominant React meta-framework	GA — Production Ready

© 2025 LoyaltyOS Inc. All rights reserved.

Kubernetes + Helm	K8s 1.30 with sidecar containers as stable feature; Helm 3.x standard	CNCF survey 2024: 84% of organizations using K8s in production	CNCF Graduated — Standard
Istio Service Mesh	Istio Ambient Mesh (sidecar-less) GA 2024 — reduces overhead 30%+	CNCF Graduated; Google, IBM, Lyft; default mesh for GKE Autopilot	CNCF Graduated
OpenTelemetry	OTel 1.0 stable; CNCF's unified observability standard replacing proprietary SDKs	CNCF Graduated; all major cloud providers and APM vendors support OTel natively	CNCF Graduated — Standard
HashiCorp Vault	Vault Secrets Operator for K8s native integration; dynamic credentials	Enterprise standard for secrets management; IBM, Salesforce, Capital One	GA — Enterprise Standard
MLflow + Feast	MLflow 2.x unified ML lifecycle; Feast 0.38 online/offline feature store	Databricks (MLflow), LinkedIn (Feast), Shopify, Twitter	GA — Maturing Standard
LLM Integration (OpenAI / Claude)	GPT-4o (May 2024) and Claude 3.5 Sonnet represent state-of-the-art for enterprise LLM tasks	Salesforce Einstein, Microsoft Copilot, ServiceNow — enterprise LLM native	Rapidly Maturing
ArgoCD (GitOps)	ArgoCD 2.10 with Application Sets; CNCF standard for GitOps CD	CNCF Graduated; Intuit, Ticketmaster, Red Hat default CD tool	CNCF Graduated
Resilience4J	Native Spring Boot 3.x integration; successor to Hystrix (deprecated)	De-facto resilience library for Spring Boot applications	GA — Production Ready

© 2025 LoyaltyOS Inc. All rights reserved.

§8 RISK REGISTER

Risk ID	Risk Description	Probability	Impact	Mitigation
R-001	Reward Engine double-issuance due to Kafka rebalance during processing	**Medium**	**Critical**	Exactly-once Kafka semantics + Redis idempotency key check before every DB write
R-002	SpEL injection attack via malicious rule configuration by compromised admin account	**Low**	**Critical**	SpEL sandbox with whitelist of allowed operations; no T() operator; MFA on admin accounts
R-003	Multi-tenant data leakage via misconfigured PostgreSQL RLS or missing tenantId filter	**Low**	**Critical**	Automated RLS policy testing in CI; tenantId scoping enforced at ORM layer with unit tests
R-004	Kafka consumer lag accumulation during traffic spikes causing reward delivery delays	**High**	**High**	KEDA autoscaling with proactive burst capacity; pre-warming strategy for known peak events
R-005	LLM-generated rules contain incorrect logic that awards excessive points	**Medium**	**High**	LLM rules always enter DRAFT; mandatory admin review; rate limits on daily point issuance per tenant
R-006	ClickHouse analytical query load impacting transactional database performance	**Medium**	**Medium**	Strict read/write separation; ClickHouse ingests from Kafka, never from PostgreSQL directly
R-007	Vendor lock-in on managed Kafka (Confluent/MSK) affecting cost at scale	**Low**	**Medium**	Abstract Kafka dependency behind internal event bus interface; evaluate Apache Pulsar as alternative
R-008	GDPR right-to-erasure conflicting with immutable ledger requirement	**High**	**Medium**	Pseudonymization: replace PII with internal UUID on erasure request; retain aggregates; legal review

© 2025 LoyaltyOS Inc. All rights reserved.

§9 GLOSSARY

Term	Definition
AST	Abstract Syntax Tree — a tree representation of the abstract syntactic structure of source code or, in this context, a rule condition tree.
BRMS	Business Rules Management System — software for creating, editing, deploying, and managing business rules (e.g., Drools).
Condition Tree	A hierarchical representation of logical conditions in a rule definition. Leaf nodes are individual conditions; internal nodes are AND/OR/NOT operators.
DLQ	Dead Letter Queue — a Kafka topic that receives messages that cannot be processed after maximum retry attempts.
EAV	Entity-Attribute-Value — a data model pattern for storing dynamic, schema-flexible attributes as rows rather than columns.
EOS	Exactly-Once Semantics — a Kafka guarantee that each message is processed exactly once even in the presence of failures.
Idempotency Key	A unique key associated with a request or command that allows the system to detect and ignore duplicate processing attempts.
KEDA	Kubernetes Event-Driven Autoscaler — CNCF project that scales Kubernetes workloads based on external event sources such as Kafka consumer lag.
Loyalty Wallet	A customer's consolidated view of their loyalty assets: points balance, tier status, badges, and active vouchers.
Multi-Tenancy	An architecture where a single software instance serves multiple customers (tenants) with strict logical data and operational isolation.
Points Ledger	An immutable, append-only record of all point credits and debits for a customer. Balance is computed as a running sum.
Reward Command	A structured, idempotent Kafka message produced by the Rule Engine and consumed by the Reward Engine, instructing it to issue a specific reward.
RLS	Row-Level Security — a PostgreSQL feature that restricts which rows a database user can access based on security policies.
SpEL	Spring Expression Language — a runtime expression evaluation language in the Spring Framework, supporting complex expressions, arithmetic, and reflection.
Tenant	A business entity (client) that has onboarded onto the LoyaltyOS platform and operates one or more loyalty programs for their end customers.
TTL	Time-To-Live — a configurable duration after which a cached value, Redis key, or points ledger entry expires and becomes invalid.

© 2025 LoyaltyOS Inc. All rights reserved.

— END OF DOCUMENT —

LoyaltyOS Platform | BRD & SRS v1.0.0 | Confidential

© 2025 LoyaltyOS Inc. All rights reserved.